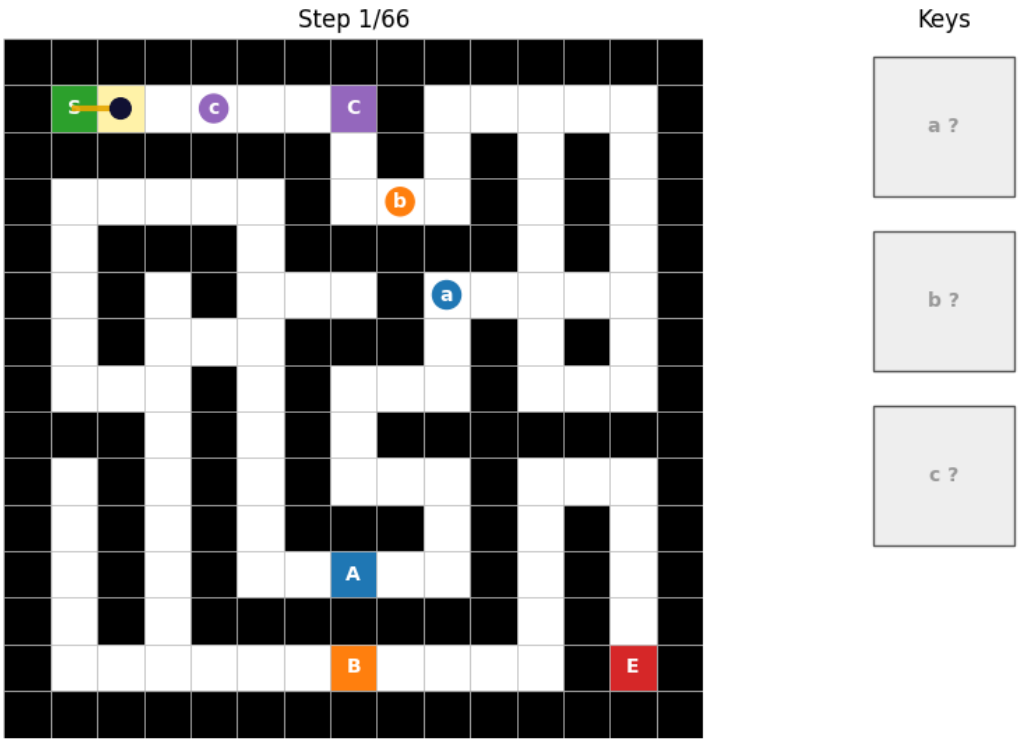
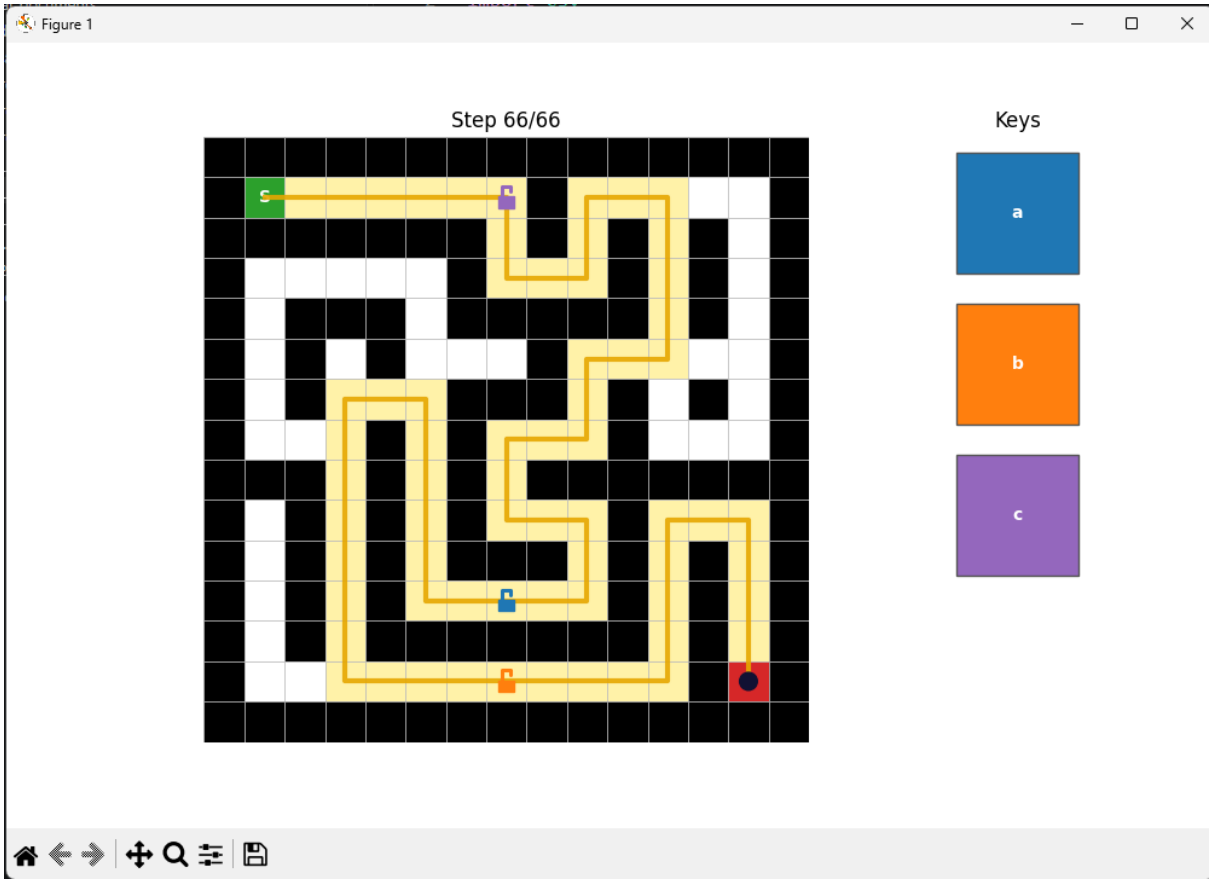
Problem 1: Output

```
Problems  Output  Debug Console  Terminal  Ports  Azure                                                    Code
PS C:\Users\shash\OneDrive\Documents\python\Hackathon> python -u "c:\Users\shash\OneDrive\Documents\python\Hackathon\maze_no_comments.py"
Enter T (1-6): 3
generated maze saved to maze.csv
66
marked path written to solution.csv
PS C:\Users\shash\OneDrive\Documents\python\Hackathon>
```





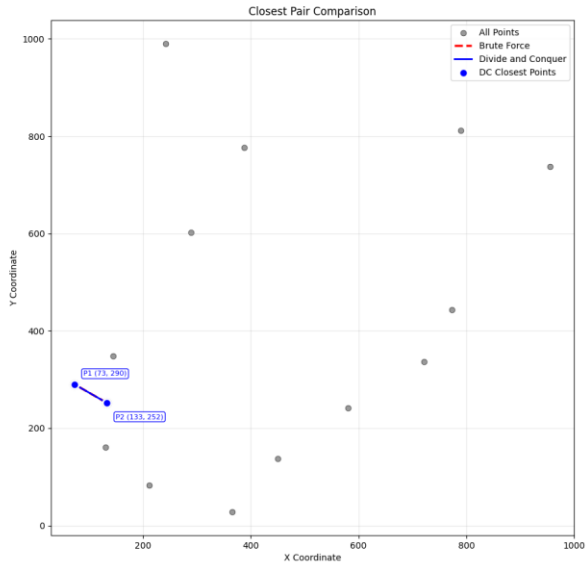
Problem 2 output

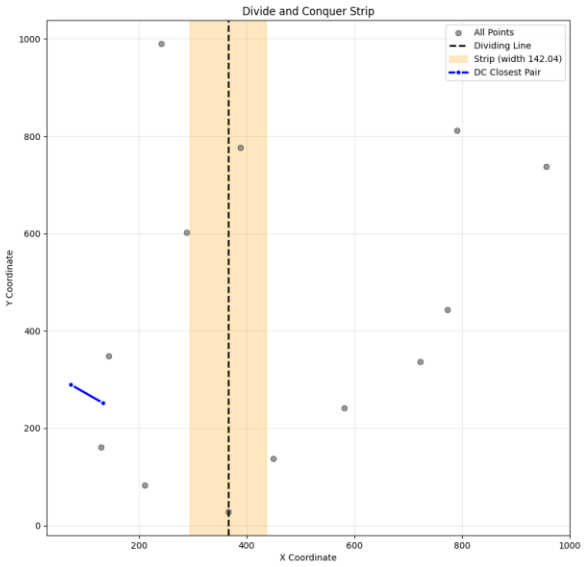
```
PS C:\Users\shash\OneDrive\Documents\python\Hackathon> python -u "c:\Users\shash\OneDrive\Documents\python\Hackathon\No_Idea\Problem_2.py"
Enter number of points (N): 15

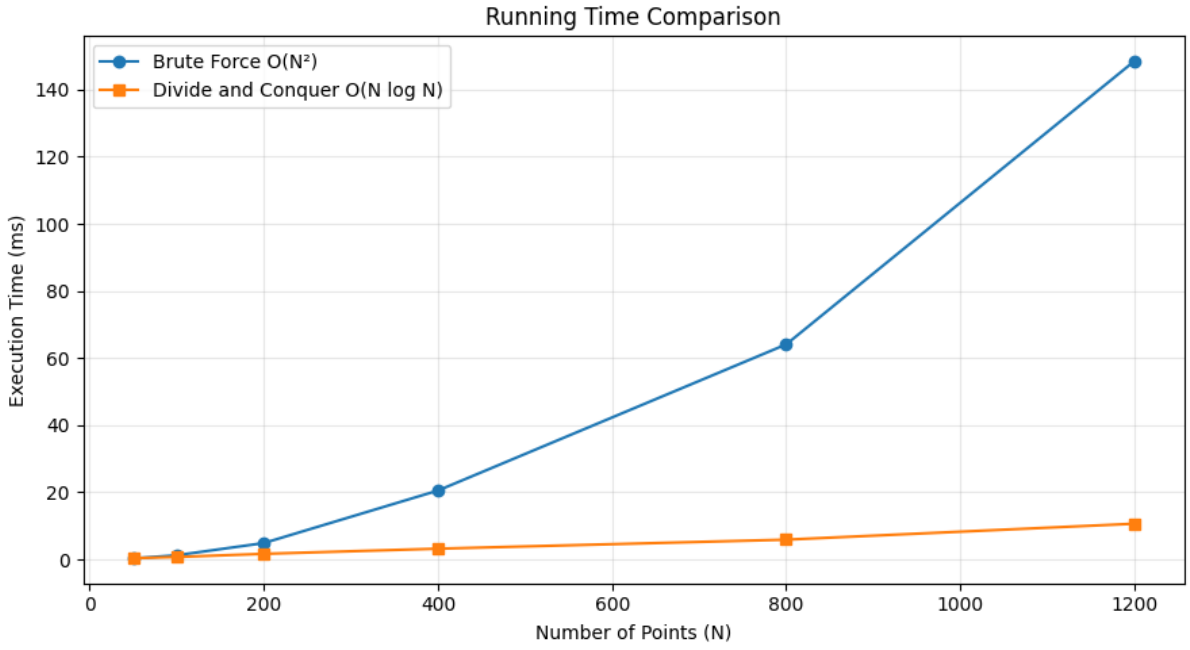
Generated Points:
(450, 138)
(956, 737)
(133, 252)
(144, 348)
(581, 241)
(73, 290)
(366, 28)
(722, 337)
(211, 83)
(242, 990)
(388, 777)
(790, 812)
(130, 161)
(773, 443)
(289, 602)

RESULTS
--------------------------------------------------
Brute Force: 71.021124 (133, 252) (73, 290)
Divide and Conquer: 71.021124 (73, 290) (133, 252)
Execution time: Brute Force = 0.041200 ms, Divide and Conquer = 0.117800 ms
Verification: Both algorithms agree.

PERFORMANCE ANALYSIS
----------------------------------------------------------------------
N         Brute Force (ms)       Divide and Conquer (ms)
50        0.289100               0.359700
100       1.241800               0.724400
200       4.875100               1.664100
400       20.525300              3.197800
800       64.071500              5.889300
1200      148.498500             10.640600
```


Closest Pair Comparison
All Points
Brute Force
Divide and Conquer
DC Closest Points
P1 (73, 290)
P2 (133, 252)
X Coordinate
Y Coordinate


Divide and Conquer Strip
All Points
Dividing Line
Strip (width 142.04)
DC Closest Pair
X Coordinate
Y Coordinate


Running Time Comparison
Brute Force O(N²)
Divide and Conquer O(N log N)
Execution Time (ms)
Number of Points (N)

Program 3:

```
========================================
   BUDGET TRAVELLER WITH COUPONS
========================================

Select an input option:
1. Enter flights manually
2. Load flights from a CSV file
3. Generate a random flight graph
Enter your choice (1-3): 1
Enter the number of cities (N): 4
Enter the number of flights (M): 4
Enter maximum intermediate stops (K): 1
Enter available coupons (C): 1

Enter the flight details.
For each flight, enter:
source  destination     price
Example:
1       2       500

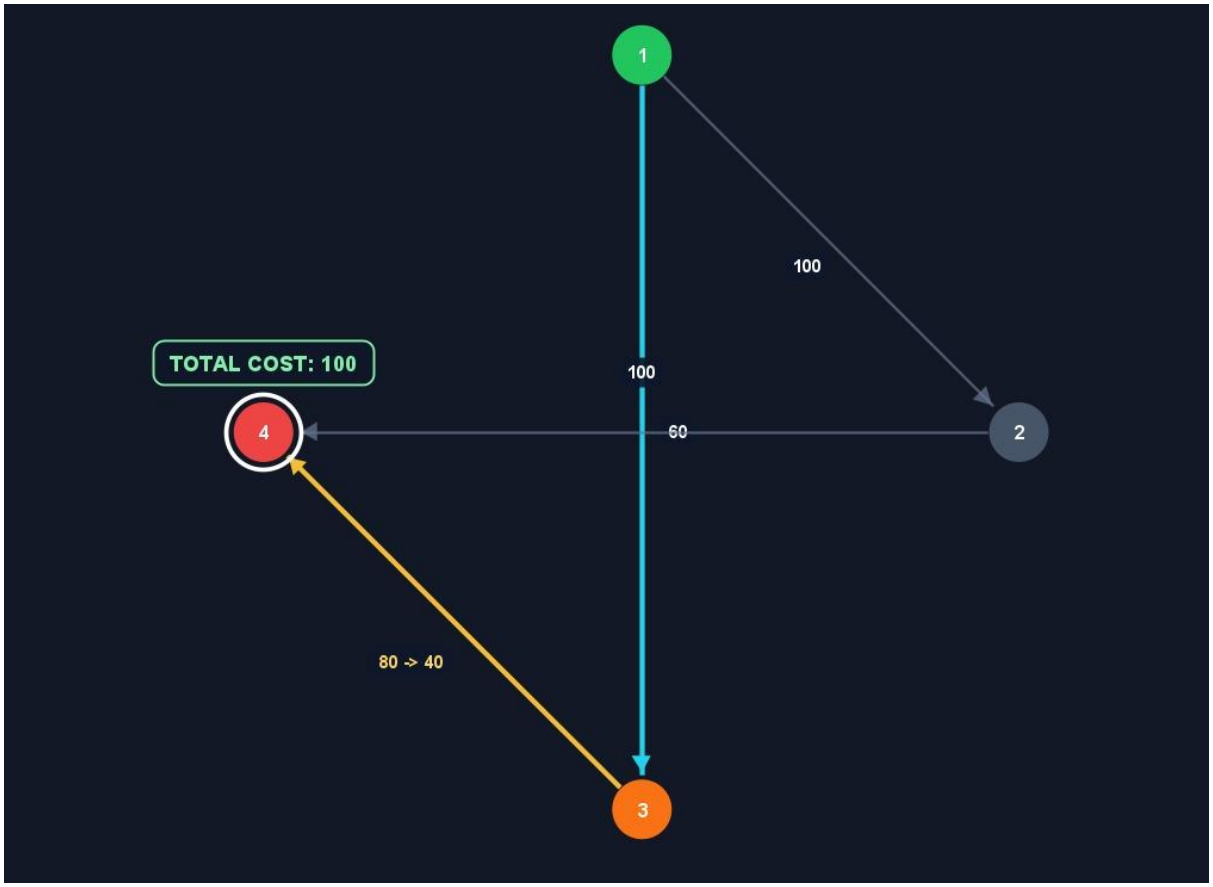
Flight 1 of 4
  Source city: 1
  Destination city: 2
  Ticket price: 100
Flight 2 of 4
  Source city: 1
  Destination city: 3
  Ticket price: 60
Flight 3 of 4
  Source city: 2
  Destination city: 4
  Ticket price: 100
Flight 4 of 4
  Source city: 3
  Destination city: 4
  Ticket price: 80
```

```
Calculating minimum travel cost...

========== OUTPUT 1 ==========
Minimum cost: 100
Selected route: 1 -> 3 -> 4
Flights used: 2
Intermediate stops used: 1
Coupons used: 1

Selected flight details:
1 -> 3 | Original price: 60 | Paid: 60
3 -> 4 | Original price: 80 | Paid: 40 | COUPON USED

========== OUTPUT 2 ==========
CSV file created: flights_output.csv
Selected flights are marked with 'x'.
Coupon usage and discounted prices are included.
```



Problem 4:

```
PS C:\Users\shash\OneDrive\Documents\python\Hackathon> python -u "c:\Users\shash\OneDrive\Documents\python\Hackathon\exam_timetable_updated.py"
Option 1: enter N M R followed by M conflict pairs
Option 2: enter N R and generate a random conflict graph
Choose option (1 or 2): 1
Enter N M R: 6 7 2
Conflict 1 (u v): 1 2
Conflict 2 (u v): 1 3
Conflict 3 (u v): 2 3
Conflict 4 (u v): 3 4
Conflict 5 (u v): 4 5
Conflict 6 (u v): 4 6
Conflict 7 (u v): 5 6

Number of courses: 6
Number of conflicts: 7
Maximum exams per slot: 2

==========================================
Welsh Powell Greedy
==========================================
Slots used: 3
Proven optimal: Yes
Slot 1: 3, 5
Slot 2: 1, 4
Slot 3: 2, 6
CSV saved: welsh_powell_timetable.csv

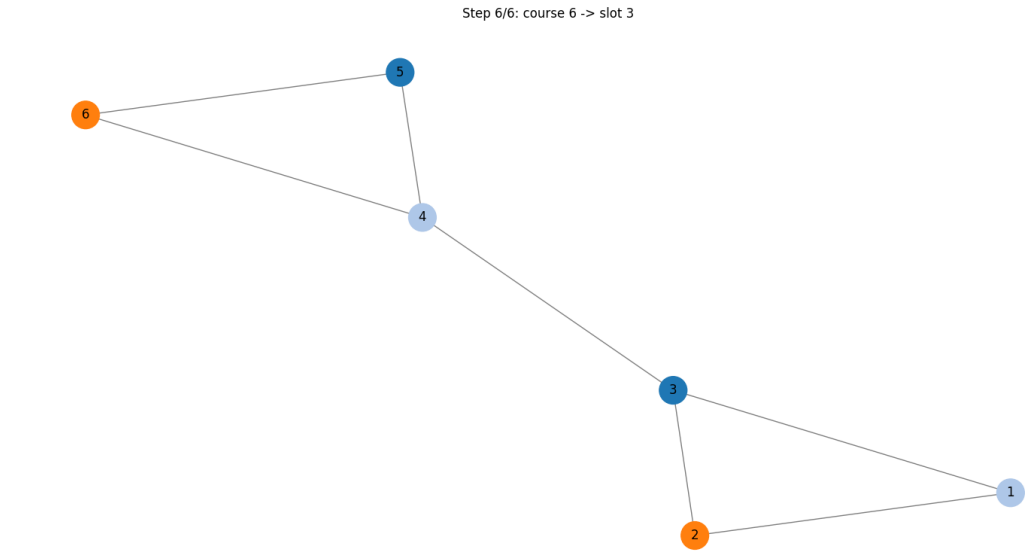
==========================================
Exact Backtracking
==========================================
Slots used: 3
Proven optimal: Yes
Slot 1: 3, 5
Slot 2: 1, 4
Slot 3: 2, 6
CSV saved: exact_backtracking_timetable.csv

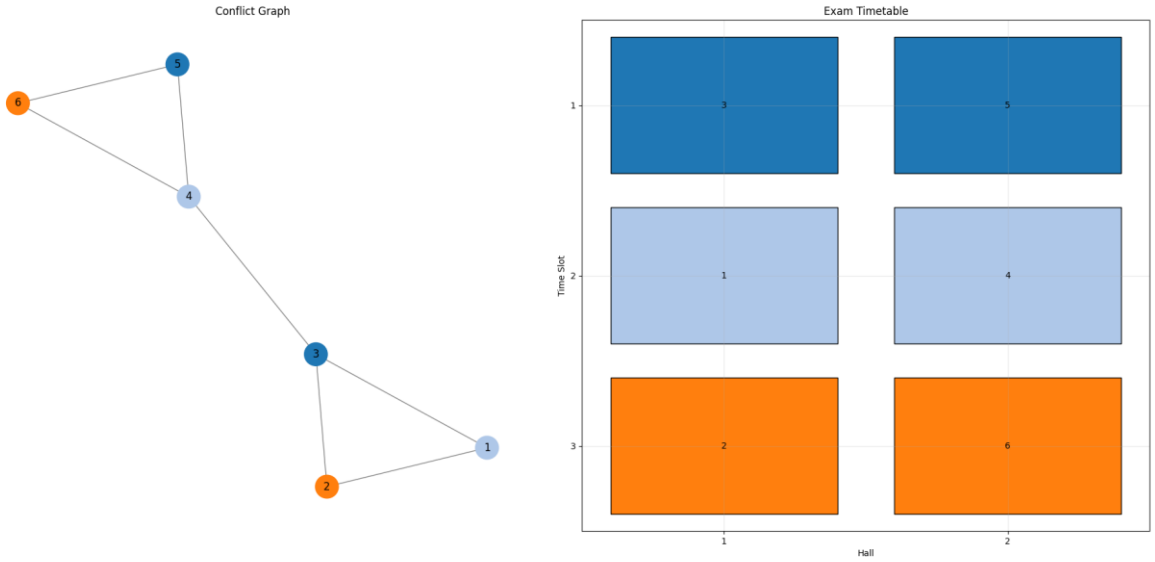
==========================================
Comparison
==========================================
Lower bound on slots: 3
Welsh-Powell : 3 slots, 0.000058s, proven optimal: Yes
Backtracking : 3 slots, 0.000007s, proven optimal: Yes
```

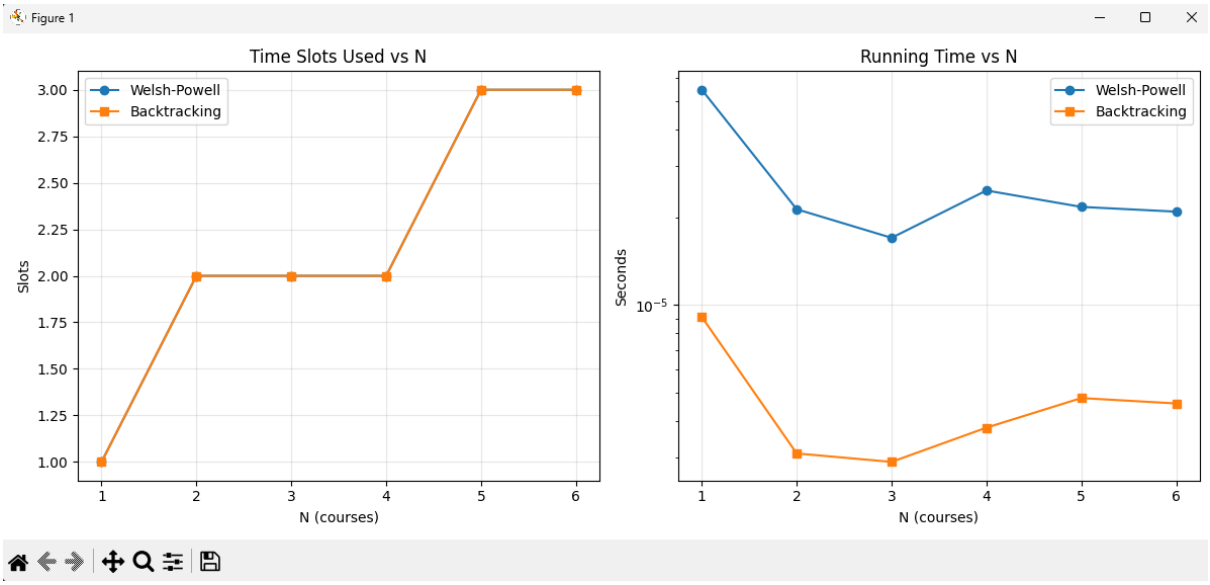
```
==============================================
Comparison
==============================================
Lower bound on slots: 3
Welsh-Powell : 3 slots, 0.000058s, proven optimal: Yes
Backtracking : 3 slots, 0.000007s, proven optimal: Yes

Animate Welsh-Powell assignment order? (y/n): y
Run scaling comparison as N increases? (y/n): y

Scaling comparison (density 0.47, R = 2)
    N  WP slots    WP time  BT slots    BT time  Optimal
    1         1   0.000055         1   0.000009      Yes
    2         2   0.000021         2   0.000003      Yes
    3         2   0.000017         2   0.000003      Yes
    4         2   0.000025         2   0.000004      Yes
    5         3   0.000022         3   0.000005      Yes
    6         3   0.000021         3   0.000005      Yes
```


Step 6/6: course 6 -> slot 3
5
6
4
3
1
2


Conflict Graph
5
6
4
3
1
2
Exam Timetable
Time Slot
Hall
1
2
3
3
5
1
4
2
6


Figure 1
Time Slots Used vs N
Welsh-Powell
Backtracking
Slots
N (courses)
Running Time vs N
Welsh-Powell
Backtracking
Seconds
10^{-5}
N (courses)